

Computer-Use Automation System

Implementation and Design Plan - interface.ai Engineering Take-Home

Assignment	Take-Home Project: Computer-Use Automation System (interface.ai)
Deliverable	Design document justified against the published evaluation criteria, revised to describe the system as built
Stack	TypeScript + Node.js 22, Playwright, LLM provider adapter (Anthropic + OpenAI), Zod
Target surface	Locally built mock credit-union back-office app (deliberately legacy-hostile), plus a re-skinned second-tenant variant
Date	September 2026 (as-built revision: September 13)
Status	As-built v1.1 - implementation complete; 93/93 tests green across 14 files

The through-line this design is built around

"The model discovers. The artifact becomes a reusable capability. Deterministic replay is how the AI agent invokes it in production."

Every component below exists to serve one of those three phases - or to keep them safe (policy), observable (evidence), or recoverable (escalation).

How to read this document. Each major decision is presented as: the decision, why it wins, the alternatives it beat and why they lose, and the evaluation criterion it serves. Section 2 maps the whole design onto the grading rubric; Sections 5-8 are the load-bearing pieces (artifact schema, deterministic replay, escalation, safety); Section 14 records the build plan as executed. This revision replaces the pre-implementation baseline: schemas, commands, and examples below are taken from the shipped repository, and places where implementation experience overturned the original design are called out explicitly (notably the redirect backstop, Section 8.5).

Contents

1. Executive summary	4
1.1 Positioning against the brief	4
1.2 Decisions at a glance	4
1.3 What is demonstrated (committed evidence and tests)	4
2. Requirements digest and evaluation mapping	6
2.1 Core requirements coverage (brief Section 3)	6
3. System architecture	7
3.1 Modules and dependency rules	7
3.2 The three data flows	8
3.3 Why a modular monolith (and not services)	8
4. Technology decisions and rejected alternatives	9
4.1 The discovery loop, concretely	9
4.2 Where typed inputs, outputs, and outcomes come from	10
5. Artifact schema - the capability contract	11
5.1 Shape	11
5.2 A step, annotated	11
5.3 Target descriptors: ranked, semantic-first locator strategies	12
5.4 Conditions: a small declarative AST	12
5.5 Parameterization and secrets	13
5.6 Versioning and review lifecycle	13
5.7 Alternatives rejected	13
6. Deterministic replay and the error taxonomy	14
6.1 Execution algorithm	14
6.2 Wait and checkpoint strategy	14
6.3 The three-class taxonomy, applied to the brief's own conditions	14
6.4 Result contract	15
6.5 What determinism means here	16
7. Escalation and human-in-the-loop handoff	17
7.1 Detecting "stuck"	17
7.2 Control-ownership state machine	17
7.3 The intervention request	17
7.4 Taking control of the live session	18
7.5 Handback and resume semantics	18
7.6 Tamper-evident dispositions (built)	19

8. Safety and data handling	20
8.1 One chokepoint, physically unavoidable	20
8.2 The allowlist model (deny by default)	20
8.3 Risky and irreversible actions	20
8.4 Redaction pipeline (regulated financial data)	20
8.5 The network-layer backstop (added during implementation)	20
8.6 Prompt-injection stance and honest limits	21
8.7 policy.yaml (excerpt; committed file in Appendix C)	21
9. Evidence and observability	22
10. Heterogeneity and multi-tenant reuse	22
10.1 The surface seam	22
10.2 Multi-tenant reuse: base artifact + tenant binding overlay (built)	23
10.3 Detecting and managing drift	24
11. The mock target application ("CU BackOffice")	25
11.1 Flows	25
11.2 Fault-injection matrix (armed one-shot via GET /__faults?arm=...)	25
12. Testing strategy - where it counts	26
13. Repository layout and deliverables mapping	26
13.1 Demo path (condensed from README)	27
14. Implementation roadmap as executed, cut lines, and stretch goals	28
14.1 Phases (as planned - and as executed; est. 28-36 focused hours)	28
14.2 Cut lines - deliberate, with the seam designed	28
14.3 Stretch goals - both attempted, both landed	28
14.4 What gets built next with more time	29
15. Risk register	29
Appendix A - The committed artifact (capabilities/member-savings-lookup.json)	30
Appendix B - Result contract examples (committed runs)	33
Appendix C - policy.yaml	34

1. Executive summary

This document describes **Scribe**, a small but complete computer-use automation system built for the interface.ai take-home. Everything described here is implemented: the claims below are backed by the committed test suite (93 tests across 14 files, all green) and by committed evidence runs. Scribe does four things end to end: (1) an LLM-driven agent accomplishes a natural-language goal against a live UI; (2) a **Recorder** distills that successful run into a typed, versioned, human-reviewable **capability artifact**; (3) a **replay engine** executes that artifact deterministically - no LLM in the decision loop - with an explicit error taxonomy separating business outcomes, recoverable conditions, and hard failures; and (4) when it cannot proceed safely, it **escalates to a human** who takes control of the same live browser session and hands control back.

The system is a **modular monolith in TypeScript**: one package, ten modules with enforced dependency boundaries, driven by a CLI plus a minimal local operator console. It automates a locally built, deliberately legacy-style **mock credit-union back-office** so that the exact runtime failures the brief cares about (record not found, validation errors, permission denials, session expiry, interstitial dialogs, slow loads) can be reproduced on demand and demonstrated in committed evidence. A re-skinned second-tenant variant of the same app ("CU North TellerWorks": different branding, labels, and origin) exists to prove artifact reuse across tenants without re-recording.

1.1 Positioning against the brief

The brief is explicit about where the difficulty lives: the artifact schema and replay contract are "a focal point of the evaluation", conflating business outcomes with failures is "the most common design mistake", and escalation must be "real, not just a TODO". This design therefore spends its complexity budget on three load-bearing pieces - the artifact schema (Section 5), the replay engine and its deviation classifier (Section 6), and the control-transfer model (Section 7) - and deliberately keeps everything else thin: no services, no queues, no database, filesystem JSON stores, a single-page operator console.

1.2 Decisions at a glance

Axis	Decision	One-line rationale
Language / runtime	TypeScript on Node.js 22	The rubric's focal point is a typed schema; Zod gives one source of truth for static types and runtime validation.
Automation substrate	Playwright (Chromium, headed)	Actionability auto-waits, first-class iframes, accessibility-tree snapshots, CDP access for live handoff, built-in tracing.
Perception	Accessibility-tree-first hybrid (+ screenshots, bbox fallback)	Works when the DOM is hostile; same abstraction exists on desktop; the brief says to bias toward no-clean-DOM approaches.
LLM	Provider adapter: Anthropic (default) + OpenAI	Tool-calling loop with strict JSON action schemas; provider is a seam, not a dependency.
Target app	Local mock CU back-office with fault injection	Full control over the error states the rubric grades; offline for reviewers; banking-realistic flows.
Architecture	Modular monolith + CLI + local operator console	The brief explicitly does not reward scaling infrastructure; boundaries live in module contracts.
Artifact store	Versioned JSON files in git	Reviewable in PRs (the review workflow IS the approval workflow); zero setup for reviewers.

1.3 What is demonstrated (committed evidence and tests)

- **Discovery**: real LLM-driven runs (committed provenance: OpenAI gpt-4o) completing "Look up member 12345 and read their current savings balance" and two further goals against the live mock app - three committed capabilities: member-savings-lookup, member-standing-check, subaccount-open.
- **Replay (happy path)**: each saved artifact re-run with params, no LLM in the process, checkpoint-verified, returning typed (and redacted) outputs.

- **Replay (business outcomes):** memberId=99999 returns MEMBER_NOT_FOUND and memberId=66666 returns ACCESS_DENIED as legitimate, structured results - not errors - alongside success variants (Active and Dormant standing).
- **Replay (recovery):** an injected interstitial dialog is detected and dismissed by a declared recovery; a slow load is retried within bounds - both visible in committed telemetry.
- **Escalation:** an injected session expiry exhausts recovery, raises an intervention, a human takes over the live browser, logs in manually, hands back with a cryptographically signed disposition, and the run resumes and completes - all captured in evidence.
- **Risky-action approval:** the mutating sub-account-open submit pauses for operator approval; a single-use approve_once lets exactly one act attempt proceed, and the run finishes with a confirmation number.
- **Cross-tenant reuse:** the identical artifact replays against the re-skinned CU North tenant (different origin, branding, and vocabulary) via a thin binding overlay - no re-recording, committed as evidence.
- **Safety:** allowlist enforcement at the act() chokepoint plus a network-layer backstop that aborts page-initiated off-allowlist traffic (including multi-hop redirect chains); redaction of sensitive values in artifacts, logs, and screenshots.

2. Requirements digest and evaluation mapping

The graders state their weighting order explicitly. The table below maps each criterion to the design element that answers it and the section that argues it. This is the contract this document is written against.

#	Evaluation criterion (in stated weight order)	Design answer	Where
1	System design - boundaries, data models, trade-offs, simplicity; artifact schema and replay contract are central	Modular monolith with three seams (surface, provider, policy); artifact split into contract / mechanics / policy; typed result contract	Sec. 3, 5
2	Correctness of the core loop - agent completes a real goal; artifact replays deterministically and verifies success	Tool-calling discovery loop; Recorder distillation; checkpoint-verified replay with no LLM import path	Sec. 4, 6
3	Robustness and error handling - runtime errors vs business outcomes vs hard failures; locator, wait, checkpoint strategy	Declared outcomes + declared recoveries + fail-closed hard failures, classified in strict precedence; ranked locators; bounded condition waits	Sec. 6
4	Human-in-the-loop escalation - detect stuck, route with context, transfer the live session, resume	Control-ownership state machine (AGENT/PAUSED/HUMAN); intervention store; same headed browser operated by the human; recorded human actions; disposition-based resume	Sec. 7
5	Generalization - heterogeneous surfaces; artifact reuse across tenants without rebuilds	SurfaceDriver seam + surface-agnostic TargetDescriptor and condition AST; base artifact + per-tenant binding overlay; drift telemetry	Sec. 10
6	Safety and data handling - allowlist, risky actions, redaction	PolicyEngine chokepoint inside the driver; risk classes with approval gates; parameterization strips PII from artifacts; masked logs and screenshots	Sec. 8
7	Code quality - readable, typed, tested where it counts	Strict TS + Zod; dependency-boundary lint; tests focused on schema, policy, classifier, locators	Sec. 12
8	Communication - reasoning, trade-offs, cut lines	REPORT.md with the seven required headings; explicit cut lines	Sec. 13, 14

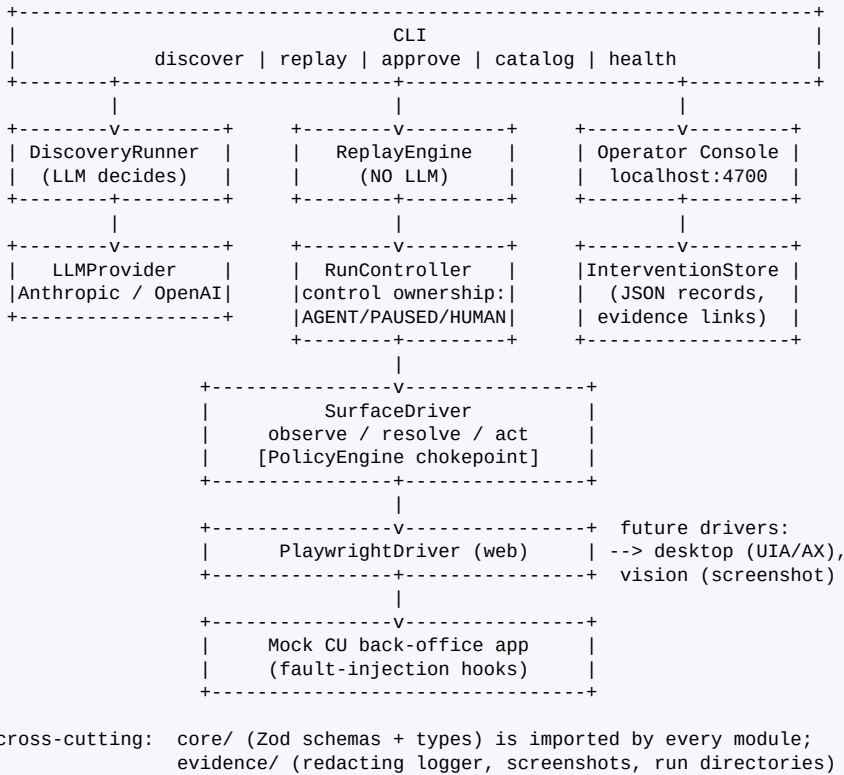
2.1 Core requirements coverage (brief Section 3)

Requirement	Coverage in this design
3.1 Goal-driven agent loop	Discovery runner: observe (a11y snapshot) - decide (LLM tool call) - act (driver), with stop conditions: goal met, max steps, timeout, loop detection, model-declared stuck.
3.2 Structured artifact	CapabilityArtifact v1: typed inputs/outputs, ranked target strategies, per-step checkpoints, declared outcomes and recoveries, policy block, provenance. Section 5.
3.3 Deterministic replay	Replay engine with per-step algorithm, deviation classifier, four-state result contract, bounded waits, fail-closed ambiguity handling. Section 6.
3.4 Safety guardrails	Deny-by-default allowlist, risk classes with conservative gates, no secrets or raw PII persisted. Section 8.
3.5 Evidence / observability	Per-run JSONL event log with actor attribution + screenshots at steps, checkpoints, and failures. Section 9.
3.6 Escalation and handoff	Intervention requests with context; live session control transfer; human-action recording; resume semantics. Section 7.
3.7 Heterogeneity and scale	Surface abstraction (design) and multi-tenant overlay model with a working cross-tenant replay against a second tenant app (built); drift telemetry + locator-health report (built). Section 10.

3. System architecture

Serves evaluation criteria: System design (criterion 1) - clear boundaries, appropriate simplicity.

Scribe is a single TypeScript package organized as a **modular monolith**: ten modules with explicit, lint-enforced dependency rules, wired together by a CLI. There are no services, queues, or databases - by design, and the brief says plainly that building scaling infrastructure is not rewarded. The boundaries that matter for the real environment exist as **interfaces** (seams), each aligned to an axis of change: the surface you automate, the model provider you call, and the policy you enforce.



3.1 Modules and dependency rules

Module	Responsibility	May depend on
core/	Zod schemas + inferred types: artifact, result contract, condition AST, policy config, intervention records. Pure data.	nothing
surface/	SurfaceDriver interface + PlaywrightDriver. Observation (a11y snapshots), target resolution, actions. Hosts the policy chokepoint.	core, policy, evidence
llm/	LLMPProvider interface + Anthropic and OpenAI implementations; strict JSON tool schemas for agent actions.	core
agent/	Discovery loop (observe-decide-act), stop conditions, Recorder (distills run into an artifact, parameterizes literals).	core, surface, llm, policy, evidence, escalation
replay/	Deterministic executor: waits, resolution, checkpoints, DeviationClassifier, recoveries, result contract. No import path to llm/ - determinism by construction, enforced by lint.	core, surface, policy, evidence, escalation
policy/	PolicyEngine: allowlist, action-kind rules, risk classification, approval gates.	core
escalation/	RunController (control-ownership state machine), intervention store, operator console server, human-action recorder.	core, evidence

Module	Responsibility	May depend on
evidence/	Redacting structured logger (JSONL), screenshot capture, run directory management.	core
target-app/	Mock CU back-office (Express + EJS) with fault-injection hooks. Runs standalone; no imports from the system.	nothing
cli/	Command wiring: discover, replay, approve, catalog, health.	all

3.2 The three data flows

- **Discovery:** goal + example params in - the runner loops (snapshot - LLM decision - policy check - act - record) until the model marks success or a stop condition fires; the Recorder then distills the action trace into a CapabilityArtifact, replacing literal values with parameter references, and writes it to capabilities/ as reviewable JSON.
- **Replay:** artifact + params in - preflight (validate params with Zod, load policy, verify app fingerprint) - per-step execute-and-verify - on deviation, classify (outcome / recovery / hard failure) - typed result out. No model calls anywhere on this path; it runs with no API key configured.
- **Escalation:** any phase can raise an intervention - the RunController flips control to PAUSED and parks the automation on a control mutex - the operator claims it in the console, drives the same headed browser, and resumes with a disposition - the engine re-verifies state and continues, with every human action recorded in the same evidence stream.

3.3 Why a modular monolith (and not services)

Decision: one process per run, seams as interfaces, filesystem stores

Why. A discovery or replay run is a single, short-lived, stateful session bound to one live browser - there is no concurrency problem for a take-home to solve, and the rubric explicitly withholds credit for queues, clusters, and multi-tenant plumbing. Every boundary that would matter at scale is expressed as a TypeScript interface with a dependency rule, which is what actually gets evaluated: can the design grow without rewrites?

Rejected: microservices / job queue. Adds deployment and IPC complexity that demonstrates nothing the rubric measures; hides the seams inside network plumbing instead of readable contracts.

Rejected: SQLite/Postgres for artifacts and interventions. A database adds reviewer setup friction and makes artifacts less reviewable. JSON files in git mean an artifact diff is a PR diff - which doubles as the review/approval workflow the artifact lifecycle needs (Section 5.6).

Production path (documented, not built). Each module maps 1:1 to a service later: driver pool per tenant, artifact registry, intervention queue + operator UI. The interfaces are the migration plan.

4. Technology decisions and rejected alternatives

Serves evaluation criteria: System design, correctness of the core loop, code quality.

Decision	Why it wins	Rejected alternatives and why
TypeScript + Node.js 22	The artifact schema is the rubric's focal point: Zod provides one definition yielding both static types and runtime validation of every artifact, param set, and result. Discriminated unions model the result contract natively. One language across agent, replay, console, and target app.	Python + Pydantic: entirely viable, but Playwright is TS-first, and splitting languages (Python system, JS console/app) weakens cohesion. Go/Java: slower iteration, weaker browser-automation ecosystem.
Playwright	Actionability auto-waits (visible/enabled/stable) remove the sleep-based flake that kills deterministic replay; first-class frame support (legacy framesets); accessibility snapshots for perception; CDP access underpins the live-session handoff; built-in tracing and screenshots feed evidence.	Selenium: manual wait discipline, flakier under dynamic content. Puppeteer: Chromium-only, thinner a11y and locator APIs. Computer-use / browser-agent SDKs: they hide exactly the loop, locator, and policy design this assignment evaluates, and their actions bypass our policy chokepoint.
Accessibility-tree-first perception	The brief: bias toward approaches that survive a hostile DOM. The agent observes a numbered snapshot of role/name/value/state/bbox nodes - not raw HTML - so decisions attach to semantics a human would use. The same abstraction exists on desktop (UIA/MSAA, macOS AX), which is the generalization story. Screenshots supplement when structure is ambiguous.	Raw DOM/CSS-first: brittle on server-rendered legacy markup, dead end for desktop. Pure screenshot + coordinates: universal but brittle to layout shifts and poor for review; retained only as the recorded last-resort strategy (bbox + element description).
LLM provider adapter (Anthropic default, OpenAI alternate)	Decisions come back as strict, schema-validated tool calls (click/fill/extract/...), each carrying machine-useable arguments plus human-readable intent and reasoning - which is what makes the recorded artifact reviewable. Temperature 0. The provider is a seam: one interface, two implementations, swappable by flag.	Hardcoding one vendor: cheaper but forfeits the seam. Free-text prompting + parsing: fragile, unvalidatable, and worse for recording structured steps.
Local mock CU back-office as target	The rubric grades how replay handles runtime states: not-found, validation errors, permission denials, session expiry, interstitials, slow loads. A local app makes each reproducible on demand via injection flags - so the committed evidence can show every class. Banking-shaped flows (member search, balances, sub-account open) mirror the brief's own examples. Offline and ToS-clean for reviewers; the tenant-B variant (CU North TellerWorks) is built and used in the cross-tenant demo.	Public demo sites (saucedemo, demoqa): zero build cost but cannot produce session expiry or permission denials on demand, which guts the robustness demonstration; ToS and rate-limit caveats; no domain realism.

4.1 The discovery loop, concretely

Each turn, the runner sends the model: the goal, declared input parameters (names + example values), a compact numbered accessibility snapshot (ref, role, name, value, state per node, plus URL and title), the last action's result, and optionally a screenshot. The model must respond with exactly one tool call from a closed set:

```
act(kind: navigate|click|fill|select|press, ref|url, value?, key?,
    risk: safe|risky, intent, reasoning)
extract(ref, outputName, type: string|number|money|date, sensitive?,
    extractPattern?, intent, reasoning)
declare_outcome(code, description, detectorText, reasoning)
finish(status: success|stuck, summary, reasoning)
```

Stop conditions (as built): finish(success), finish(stuck), turn budget (policy default 40), wall-clock budget (5 minutes), the same action against the same page state observed three times (loop detection), provider failure after retries, or an operator abort at a risky-action approval. declare_outcome is grounded: the engine rejects it unless the declared

detector text is actually visible on the current screen. Every executed action plus the observation that followed it is appended to the run trace that the Recorder distills. The model never executes anything itself: every action passes through the same SurfaceDriver - and therefore the same PolicyEngine - that replay uses.

4.2 Where typed inputs, outputs, and outcomes come from

- **Inputs:** declared at the CLI at discovery time (e.g. memberId=12345). The Recorder replaces exact literal occurrences in fills and URLs with `{{inputs.memberId}}` - parameterization doubles as PII stripping.
- **Outputs:** the agent's `extract(name, ref, type)` calls declare them; the Recorder captures the target descriptor and declared type into the outputs block.
- **Outcomes:** a happy-path run never sees "member not found", so outcomes come from two honest sources: **probe runs** (re-run discovery with an exemplar bad input, e.g. memberId=99999; the model calls `declare_outcome` and the CLI merges the declared outcome into the existing artifact) and **human review** of the artifact JSON, which is designed to be hand-editable. No hallucinated detectors: `declare_outcome` is rejected unless its detector text is visible on screen at the moment of declaration.

5. Artifact schema - the capability contract

Serves evaluation criteria: System design (the stated focal point), robustness, generalization, safety.

The artifact is the product of discovery and the input to replay - the brief calls it a capability an AI agent can call, and says to design the schema deliberately. The single organizing idea here: **separate the contract from the mechanics from the policy**. A calling agent needs only the contract (what to send, what comes back, which business outcomes exist). The replay engine consumes the mechanics (steps, targets, waits, checkpoints). The policy engine reads the policy block. A human reviewer reads all three - which is why every step keeps the model's stated intent as prose.

5.1 Shape

```
CapabilityArtifact (v1, as built in core/artifact.ts) // stored as capabilities/<id>.json
|- schemaVersion: "1.0" // format version -> migration path
|- capability: { id, name, description, version (semver) }
|- target: { appId, surface: "web", endpoint (templated URL),
|           appFingerprint: { titlePattern?, markers[] } } // fail fast on drift
|- inputs[]: { name, type, required, pattern?, enumValues?, sensitive,
|             description?, example? }
|- outputs[]: { name, type, sensitive, sourceStep, description? }
|- steps[]: { id, intent, action, target?, url?, value?, key?, sensitive,
|            outputName?, extractPattern?, waitBefore?, checkpoint?,
|            risk: "safe"|"risky" }
|- outcomes[]: { code, description, terminal: true, detector: Condition }
|- recoveries[]: { id, appliesTo: "global"|stepIds[], detector: Condition,
|                 action: dismiss(target) | waitRetry(backoffMs) |
|                 runSteps(stepIds), maxAttempts }
|- successCheckpoint: Condition
|- policy: { requiredOrigins[], allowedActionKinds[],
|           riskLevel: "readonly"|"mutating", unattendedReplay: boolean }
|- provenance: { provider, model, runId, recordedAt, evidenceRef?,
|               reviewStatus: "draft"|"approved", reviewedBy?, reviewedAt? }

// Zod superRefine cross-checks at parse time: unique step ids, every
// outputs[].sourceStep and recoveries[].appliesTo references a real step.
```

Appendix A contains the committed artifact for member-savings-lookup, as the Recorder actually emitted it from a real discovery run. Everything is Zod-defined in core/; the static types are inferred from the same definitions that validate at runtime, so an artifact that parses is an artifact the engine can run. Three capabilities are committed: member-savings-lookup (read), member-standing-check (read, four result shapes), and subaccount-open (mutating, 14 steps through a confirmation screen).

5.2 A step, annotated

```

{
  "id": "s3",
  "intent": "Enter the member number into the search field",    // model's words, kept
  "action": "fill",
  "target": {
    "strategies": [
      { "kind": "role",      "role": "textbox", "name": "Member Number" },
      { "kind": "labelText", "value": "Member Number" },
      { "kind": "nearText",  "value": "Member Number" },
      { "kind": "css",       "value": "#ctl00_pnlSrch_txtMbr" }, // observed, brittle,
      { "kind": "bbox",      "value": [412, 236, 180, 24] }      // per-tenant patchable
    ],
    "elementDescription": "Member number input in the search panel",
    "framePath": [] // frameset-safe addressing
  },
  "value": "{{inputs.memberId}}", // literal 12345 parameterized away
  "waitBefore": { "condition": { "elementVisible": { "kind": "labelText",
    "value": "Member Number" } }, "timeoutMs": 5000 },
  "checkpoint": { "valueMatches": { "target": { "kind": "labelText",
    "value": "Member Number" }, "pattern": "{{inputs.memberId}}" } },
  "risk": "safe"
}

```

5.3 Target descriptors: ranked, semantic-first locator strategies

Each target carries an ordered list of independent strategies, tried at replay time until exactly one node matches. The ranking encodes a robustness argument, not a preference:

Rank	Strategy	Why this position
1	role + accessible name	Most stable identity across styling, branding, and markup refactors; portable to desktop accessibility APIs; it is how a human names the control ("the Member Number textbox").
2	labelText (associated label)	Explicit label-control association; very stable on server-rendered form-heavy enterprise apps.
3	nearText (anchor on visible text)	Legacy tables often have no label association at all - the visible text near the control is the only semantic anchor. Survives non-semantic markup; this is the no-clean-DOM workhorse.
4	css (as observed)	Precise and fast when it works, but brittle and tenant-specific (generated IDs). Recorded because it was true at discovery; the natural slot for per-tenant overrides (Section 10).
5	bbox + elementDescription	Universal last resort; also the bridge to screenshot/vision drivers and desktop surfaces. Description is for humans and vision models alike.

Uniqueness rule: ambiguity fails closed

A strategy that matches zero nodes or more than one node is treated as a miss and the next rank is tried; if all ranks miss, the step raises a deviation - the engine never guesses among candidates. In a banking back-office, acting on the wrong control is strictly worse than stopping. Replay telemetry records which rank matched (fallback depth), which becomes the drift signal in Section 10.3.

5.4 Conditions: a small declarative AST

Waits, checkpoints, outcome detectors, and recovery detectors all share one predicate language:

```

Condition := // core/conditions.ts, as built
  { "all": Condition[] } // conjunction
| { "any": Condition[] } // disjunction
| { "not": Condition }
| { "urlMatches": "<pattern>" } // template-aware
// ("/members/{{inputs.memberId}}")
| { "textPresent": "<string>" }
| { "elementVisible": LocatorStrategy }
| { "elementAbsent": LocatorStrategy }
| { "valueMatches": { target: LocatorStrategy, pattern } } // input assertions

```

- **Serializable and reviewable:** a checkpoint is data a reviewer can read, not code that must be trusted; no eval, no injection surface.
- **Surface-agnostic:** every predicate is expressible against web, desktop accessibility trees, or OCR/vision output - conditions do not name DOM concepts.
- **Composable:** outcome detectors are just conditions, so "no records found banner visible AND results table absent" is one declaration.

5.5 Parameterization and secrets

The Recorder rewrites every occurrence of a declared input's literal value (in fill values, URLs, and checkpoint patterns) into `{{inputs.name}}` references - which is simultaneously the reuse mechanism and the reason raw customer data never lands in an artifact. Credentials never appear even at discovery time: login steps reference `{{secrets.tellerPassword}}`, resolved from the environment at runtime, and marked so evidence masks them. Sensitive-flagged inputs and outputs are masked in every log line and screenshot (Section 8.3).

5.6 Versioning and review lifecycle

- **schemaVersion** versions the format (migrations); **capability.version** (semver) versions the flow - a locator patch is a patch bump, a changed contract (inputs/outputs/outcomes) is a major bump because callers depend on it.
- **appFingerprint** pins what the flow was recorded against; replay verifies it up front and fails fast with `DRIFT_SUSPECTED` rather than acting on a changed app.
- **reviewStatus:** `draft` -> `approved` is set by a human after reading the artifact (`npm run approve -- --capability` --reviewer stamps `reviewedBy` and `reviewedAt`; the JSON diff is a git PR review). Risky capabilities cannot replay unattended while in draft - the approval state is enforced by the policy engine, not by convention.

5.7 Alternatives rejected

Alternative	Why rejected
Record a runnable script (generated Playwright code)	Not reviewable as data, not parameterizable without parsing code, no seam for policy checks or outcome declarations, and locked to one surface technology. Code generation from the artifact is the right direction (a stretch goal), not the storage format.
Store the raw LLM transcript as the artifact	The brief explicitly requires decoupling. Transcripts are nondeterministic, unreviewable, and full of raw page content (PII). Provenance keeps a pointer to the run, not the transcript.
Single flat step list without declared outcomes/recoveries	Forces every deviation to be a failure - precisely the business-outcome/failure conflation the brief warns is the most common design mistake.
One monolithic locator per element	A single selector is a single point of failure and gives no cross-tenant story; ranked strategies degrade gracefully and make drift measurable.

6. Deterministic replay and the error taxonomy

Serves evaluation criteria: Correctness of the core loop, robustness and error handling (criteria 2-3).

Replay is the production path: given an artifact and typed params, execute the flow with **no model in the decision loop** and return a structured result. The engine's design goal is honesty under deviation - the brief is explicit that the interesting failures are runtime conditions, not layout drift, and that a replay must respond deliberately rather than blindly proceed.

6.1 Execution algorithm

```
replay(artifact, params):
    validate params against artifact.inputs (Zod)           // typed at the boundary
    policy.load(artifact.policy + global policy.yaml)
    driver.open(render(artifact.target.entrypoint))
    assert appFingerprint                                  // else DRIFT_SUSPECTED hard fail

    for step in artifact.steps:
        waitFor(step.waitBefore, bounded)                  // named condition, never sleep()
        guardScan()                                         // cheap global detectors: known
                                                             // dialogs, session-expiry marker
        handle = resolve(step.target)                      // ranked; unique-or-next-rank;
                                                             // records fallbackDepth
        driver.act(step.action, handle, render(step.value)) // PolicyEngine check inside act()
        ok = verify(step.checkpoint, bounded)
        if deviation (resolve miss | act error | checkpoint fail | guard hit):
            classify(snapshot):
                1. matches artifact.outcomes[].detector    -> return BUSINESS_OUTCOME(code)
                2. matches recoveries[].detector            -> apply recovery, retry step
                                                             (bounded by maxAttempts)
                3. else                                    -> evidence bundle;
                                                             escalate if policy allows;
                                                             else return HARD_FAILURE(step,
                                                             expected, observed, evidence)

    outputs = extract(artifact.outputs)                    // from declared source steps
    verify(artifact.successCheckpoint, bounded)            // else classify as above
    return SUCCESS(outputs, telemetry)
```

6.2 Wait and checkpoint strategy

- **No sleeps, ever.** Every wait is a named condition with a bounded timeout, recorded in the artifact from what discovery actually observed (URL changed, element appeared). Playwright's actionability checks (visible/enabled/stable) cover the last mile before each action.
- **A checkpoint after every step**, not only at the end. Rationale: it bounds the blast radius of a silent failure to one step, makes the failing step the unit of debugging (what was expected vs observed, right there), and gives the resume logic after human handoff a precise re-entry test (Section 7.5).
- **A final successCheckpoint** asserts the business goal itself (e.g. the balance panel for that member is visible) so success is verified, never assumed - the glossary's definition of a checkpoint.
- **Guard scans between steps** run a handful of cheap global detectors (known interstitial, session-expiry marker, error banner) so an unexpected dialog is caught at the seam where it appears, not three steps later as a confusing resolve failure.

6.3 The three-class taxonomy, applied to the brief's own conditions

Runtime condition	Class	Replay response	What the caller sees
Record / member not found	Business outcome	Detector matches; extract any declared fields; stop cleanly.	status=business_outcome, code=MEMBER_NOT_FOUND

Runtime condition	Class	Replay response	What the caller sees
Validation error on a form	Business outcome (declared)	Detector matches the field-error state; committed example: the sub-account form rejecting a low deposit.	code=DEPOSIT_BELOW_MINIMUM
Permission denied	Business outcome (terminal)	Legitimate state the caller must know; never retried.	code=PERMISSION_DENIED
Known interstitial dialog (e.g. maintenance notice)	Recoverable	Declared recovery dismisses it; step retried; recovery logged.	Success (telemetry shows the recovery)
Transient slowness / failed load	Recoverable	Bounded wait-and-retry with backoff (maxAttempts).	Success, or hard failure once bounds exhaust
Session timeout / expiry	Recoverable (bounded) or escalate	A declared runSteps recovery re-runs the login steps ({{secrets.*}} resolved from env); otherwise intervention.	Success after re-auth, or status=escalated
Unexpected unknown dialog	Hard failure -> escalate	No detector matches: never dismiss what we cannot name.	status=escalated (or hard_failure with evidence)
App error / HTTP 500	Hard failure	Evidence bundle (screenshot, a11y snapshot, URL, console); stop.	status=hard_failure with step, expected, observed
Ambiguous or missing target	Hard failure	All strategy ranks missed or matched multiple nodes; fail closed.	status=hard_failure (+ DRIFT_SUSPECTED hint)

Why classification runs in this exact precedence

Outcomes before recoveries: an expected business state must never be "recovered" away - retrying a not-found lookup is at best wasteful and at worst masks the caller's answer.

Recoveries before failure: only conditions someone declared and bounded (maxAttempts) are recoverable; recovery is a whitelist, not a heuristic.

Failure is the default: anything unrecognized fails closed with evidence. The classifier evaluates detectors against one stable snapshot so the decision itself is deterministic.

6.4 Result contract

```
// core/result.ts, as built: RunOutcome, a Zod discriminated union on status.
type RunOutcome =
  | { status: "success"; outputs: Record<string, string> }
  | { status: "business_outcome"; code: string; // e.g. "MEMBER_NOT_FOUND"
    description: string; extracted: Record<string, string> }
  | { status: "hard_failure"; error: { stepId?, intent?, expected: string,
    observed: string, evidence: string[] } } // paths into the run dir
  | { status: "escalated"; interventions: InterventionSummary[]; // >= 1
    finalStatus: "success" | "business_outcome" | "hard_failure" | "aborted";
    outputs?:; code?:; description?:; extracted?:; error?: };
// ^ the underlying result is carried through the escalation wrapper

// InterventionSummary: { id, reason, type: "assist" | "approval",
// disposition: fixed_environment | completed_step | approve_once | deny
// | abort | expired, operator? }
//
// The RunResult wrapper adds: runId, mode, capability id + version, params
// (sensitive values pre-masked), timestamps, evidenceDir, and per-step
// telemetry: { stepId, strategyRank (0 = primary; rising = locator drift),
// strategyKind, targeted, attempts, recoveriesApplied[], durationMs }.
// Business conditions are values, not thrown errors.
```

One deliberate property of the escalated arm: it wraps the underlying result rather than replacing it. A run that needed a human still returns its business answer - the committed escalation evidence ends in finalStatus=success with outputs, plus the intervention record of who helped and how. Callers that only care about the answer read finalStatus; callers that audit read the interventions array.

6.5 What determinism means here

Same artifact + same params + same app state produces the same action sequence and the same classification.

Controlled: no model calls (enforced by the import-boundary lint on replay/); branching only through declared detectors; fixed viewport on the browser context; template rendering is pure. Allowed to vary: timing within bounded waits and which locator rank matched - both are recorded as telemetry, not silent. UI drift - secondary per the brief - surfaces as rising fallback depth or a fingerprint mismatch and is reported as DRIFT_SUSPECTED rather than misclassified as a random failure.

7. Escalation and human-in-the-loop handoff

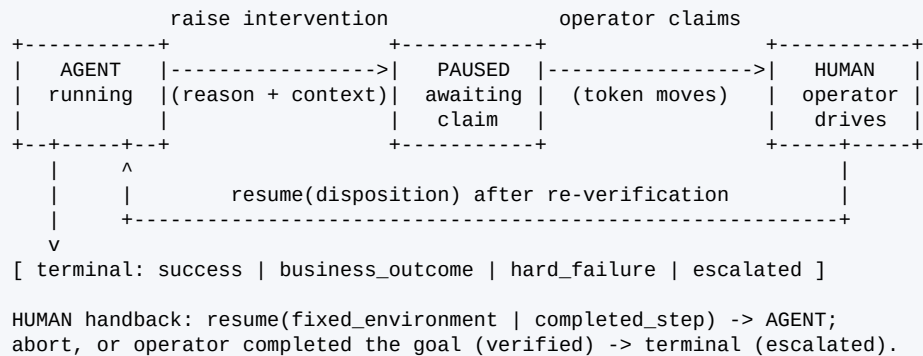
Serves evaluation criteria: Human-in-the-loop escalation (criterion 4) - a real mechanism, not a TODO.

The design centers on one idea: **control of a live session is an owned token with an auditable state machine**, not an ad-hoc pause. At any moment exactly one party may act - the automation or a named human - and every transition is logged with who, when, and why. The same mechanism serves both uses the brief names: stuck states and risky-action approvals.

7.1 Detecting "stuck"

Phase	Trigger
Discovery	Loop detection (same action + same state hash three times); max steps or wall-clock budget exceeded; the model itself calls escalate(reason); PolicyEngine blocks a risky action the model requested.
Replay	DeviationClassifier reaches class 3 (no outcome, recoveries exhausted or none apply); unknown dialog, failed checkpoint, ambiguous target, app error. Escalation is attempted before returning a hard failure when policy allows an operator.
Approval	A step marked risk=risky requires confirmation (discovery: always; replay: whenever unattendedReplay is false or reviewStatus is draft). Raised as an intervention of type approval - same pipeline, different disposition set (approve_once / deny).

7.2 Control-ownership state machine



The automation loop physically cannot act while it does not hold the token: the RunController exposes an awaitControl() gate the executor passes through before every step, and the driver tags every action with the current holder. This is the seam the brief asks about - pause, cede, resume on the same session - expressed as a mutex plus a state machine rather than convention. As built, custody history is also **tamper-evident**: every transition is written to run.jsonl as a control_transition event carrying the head of a runId-seeded SHA-256 hash chain, so editing any historical transition breaks every chain hash recorded after it.

7.3 The intervention request

```
// Committed: evidence/replay_escalation_20260912180848/interventions.json
{
  "request": {
    "id": "iv_replay_escalation_20260912180848_1",
    "runId": "replay_escalation_20260912180848",
    "type": "assist", // or "approval" for risky steps
    "capabilityId": "member-savings-lookup",
    "reason": "urlMatches(/desk) - observed: checkpoint not satisfied",
    "stepId": "s4",
    "intent": "Sign in to teller system",
    "currentUrl": "http://localhost:4600/login?expired=1",
    "screenshotPath": "screenshots/004_s4_intervention.png",
    "requestedAt": "2026-09-12T18:08:52.174Z",
    "expiresAt": "2026-09-12T18:38:52.174Z" // TTL (30 min policy default);
  }, // a claim stops the clock
  "status": "resolved", // pending -> claimed -> resolved | expired
  "claimedBy": "operator-jsmith",
  "claimedAt": "2026-09-12T18:08:52.221Z",
  "resolution": {
    "disposition": "completed_step",
    "operator": "operator-jsmith",
    "note": "re-authenticated in the live session; step's own checkpoint
            should now hold"
  },
  "resolvedAt": "2026-09-12T18:08:52.299Z"
} // resolved records also carry an HMAC signature block - Section 7.6
```

7.4 Taking control of the live session

- The browser runs **headed**; the session (cookies, storage, page state) is the one the automation was using - not a fresh context.
- The **operator console** (a deliberately minimal local web page, localhost:4700) lists open interventions with full context: capability, step and intent, reason, screenshot, current URL. Claiming one flips the state machine to HUMAN and hands the operator the token. Every console route requires a bearer token (x-scribe-token header, or ?token= in the CLI-printed URL for the dashboard), compared timing-safe; set SCRIBE_CONSOLE_TOKEN for a stable token, otherwise each run mints a fresh random one.
- The operator acts **directly in the same browser window**. An injected recorder (a page binding capturing clicks, fills, and navigations, with values redacted) writes each human action into the same evidence stream with actor="human" - so the record of what the human did survives the handoff, as required.
- Production design (documented, not built)**: for remote operators the same token model rides on CDP screencast or noVNC to a browser pool; the console gains a live view. Nothing in the artifact, engine, or state machine changes - only the transport for human input. This is the deliberate mock seam the brief invites.

7.5 Handback and resume semantics

Disposition	Meaning	Engine behavior on resume (as built)
fixed_environment	"I repaired the state (e.g. re-logged-in); the step never completed."	Checkpoint-first: if the operator's fix already restored the step's postcondition, re-acting would double-apply it, so the engine advances; otherwise the step is retried from its waitBefore.
completed_step	"I performed this step manually."	Verify the current step's checkpoint against the live session; pass means advance, fail means hard failure ("operator marked the step complete but its checkpoint still fails") - never re-execute a mutating step a human already did.
approve_once / deny	Decision on a risky-action approval request.	approve_once is consumed by exactly one executed act attempt - a retry of the same step must re-ask; deny classifies the run as blocked and stops with a clear result.
abort	"This run should not continue."	Terminal: result status escalated with finalStatus=aborted, evidence sealed.

Disposition	Meaning	Engine behavior on resume (as built)
expired	Nobody claimed the intervention before its TTL.	Fail closed: the run ends escalated with finalStatus=aborted. A claim stops the clock, so a human actively working is never timed out.

Why checkpoints make resume safe

Resume never trusts the human's claim: it trusts the artifact's per-step checkpoints, evaluated against the live session. The human says what they intended; the engine verifies what is true. This is also why per-step checkpoints (Section 6.2) are non-negotiable in the schema: they are the re-entry tests for handoff, not just failure locators.

7.6 Tamper-evident dispositions (built)

A resolution record ("this operator approved this risky step") is an accountability artifact; anyone with write access to the evidence directory could otherwise edit it after the fact. As built, every operator disposition is **HMAC-SHA256 signed** over a domain-separated, fixed-order canonical payload (domain scribe.resolution.v2) binding interventionId, runId, capabilityId, stepId, disposition, operator, note, resolvedAt, **and the control-chain head at the moment of hand-back** - so a signature for "approve step s13 of subaccount-open in run X" cannot be replayed against, or reinterpreted as, anything else, and is pinned to the exact custody history it happened under.

- **Auditor check:** verifyResolutionForRecord() recomputes the payload from the *stored* record and verifies the MAC against that - editing resolution.note or request.capabilityId in interventions.json fails verification even if the signature object itself is left untouched.
- **Key handling:** the signing key comes from SCRIBE_SIGNING_SECRET (defaulting to the console token), identified in the record by a fingerprint keyId; comparisons are timing-safe.
- **Honest scope cut:** HMAC with one shared key means verifier = trusted auditor holding the key. Production swaps in per-operator identity (OIDC) and KMS-held asymmetric keys behind the same seam; nothing outside the signing module knows the algorithm.

8. Safety and data handling

Serves evaluation criteria: Safety and data handling (criterion 6); also constrains discovery and replay design.

8.1 One chokepoint, physically unavoidable

The PolicyEngine is invoked **inside SurfaceDriver.act()** - not by the callers. Discovery, replay, and any future assisted-recovery path all act through the driver, so no code path can reach the UI without passing policy. This placement is the design's single most important safety property: enforcement by construction rather than by caller discipline. (The alternative - each runner checking policy before calling the driver - fails open the day someone adds a third caller.)

8.2 The allowlist model (deny by default)

- **Origins and routes:** navigation and actions are permitted only on origins/URL patterns listed in policy.yaml plus the artifact's requiredOrigins. Anything else is blocked and logged - during discovery the model is told its action was policy-blocked (it must find another way or escalate), during replay the block is a hard failure with evidence.
- **Action kinds:** the global config and each artifact declare which of navigate/click/fill/select/press/extract are permitted; e.g. a read-only capability that suddenly needs a submit fails policy, which catches both drift and tampering.
- **Budgets:** max steps, per-step and per-run timeouts, and max recovery attempts are policy, so runaway loops are structurally impossible.

8.3 Risky and irreversible actions

Every step carries risk: safe or risky, assigned at recording (mutating submits, anything past a confirmation screen) and reviewable. Handling is conservative and justified by asymmetry: a blocked action costs seconds of operator time; a wrong irreversible write in a banking system has unbounded cost. Concretely: during discovery, risky actions always require a human approval (the intervention pipeline); during replay, risky steps run unattended only when the artifact is reviewStatus=approved AND its policy sets unattendedReplay=true - otherwise the run pauses for approval. Reads never require approval.

8.4 Redaction pipeline (regulated financial data)

Layer	Mechanism
Artifacts	Recorder parameterization replaces literal input values with {{inputs.*}} references; sensitive example values are never stored; credentials exist only as {{secrets.*}} references resolved from the environment at runtime.
Logs	The evidence logger applies redaction before write: values of sensitive-flagged params/outputs and any {{secrets.*}} resolution are masked (never logged in the clear, anywhere) - including URL-encoded variants, so a secret embedded in a query string is caught too.
Screenshots	Steps that touch sensitive-flagged fields are captured with Playwright's mask option over those locators; a per-step screenshot=off escape hatch exists for fully sensitive screens.
LLM transcript	Not persisted. Provenance stores model, provider, runId, and distilled per-step intents - the decoupling the brief requires - so page content containing PII does not outlive the run.
Repo hygiene	No secrets in the repo: .env.example documents required keys; the mock app ships only synthetic data.

8.5 The network-layer backstop (added during implementation)

The act() chokepoint refuses to *initiate* off-allowlist actions - but a malicious or compromised page can initiate traffic by itself: tracker pixels, scripted navigations, server-side redirects. Implementation therefore added a second enforcement line at the network layer, inside the driver, and it is the one place where building the system overturned the original design:

- **Route interception:** every request in the browser context passes a context.route handler; requests to off-allowlist origins are aborted in-flight and logged as net_blocked events - the request never leaves the browser. This covers page-initiated subresources and link navigations alike.
- **Server redirects vetted at the source:** the handler fetches with maxRedirects: 0 and vets the first Location header; an off-allowlist destination is replaced with a synthetic 502 blocking response, so the browser never follows it and the run stays on the allowlisted origin.
- **The uninterceptable hop:** Playwright route handlers are never re-invoked for deeper hops of a redirect chain (verified empirically: when a handler fulfills a 3xx, the browser follows it on the direct network path). A request-stream watchdog catches any redirectedFrom() hop whose URL leaves the allowlist, records the violation, and tears the browser context down - the run fails closed. This is detection-and-kill, not prevention: one hop's egress can race the teardown, a residual that is documented and asserted as such in the integration tests rather than papered over.
- **A rejected fix, on the record:** probing chain hops from Node before letting the browser proceed looked airtight but executes every hop twice - which corrupts one-shot server state. The mock app's armed session-expiry fault fired on the probe, the browser never saw the expiry, and four integration tests failed; first-hop vetting plus the watchdog replaced it. Chain-walking a stateful server from outside the browser is a design error.
- **Side channels closed:** service workers are blocked at context creation (no script-controlled fetch layer outside the routes) and WebSockets are denied wholesale (closed with policy code 1008) - policy cannot vet a full-duplex stream, so it refuses it.

8.6 Prompt-injection stance and honest limits

- Page content is **untrusted input**: the agent's system prompt instructs the model to treat on-screen text as data, but the real guarantee is structural - whatever the model asks for, the allowlist and risk gates hold at the chokepoint. A page cannot talk the system into acting off-policy; at worst it wastes bounded steps.
- **Limits stated plainly:** redaction is flag-driven, not a DLP scanner - an unflagged sensitive field would leak into evidence until review catches it; screenshot masking is best-effort on legacy layouts; the mock app's auth is synthetic. These are documented in REPORT.md's Safety section rather than papered over.

8.7 policy.yaml (excerpt; committed file in Appendix C)

```
allowlist:
  origins:
    - http://localhost:4600 # mock CU back-office - tenant A
    - http://localhost:4650 # re-skinned tenant B (CU North)
actionKinds: [navigate, click, fill, select, press, extract]
risky:
  discoveryRequiresConfirmation: true # human approves every mutation
  unattendedRequiresApprovedArtifact: true # risky replay needs review
budgets: { discoveryMaxTurns: 40, stepTimeoutMs: 15000,
            runTimeoutMs: 300000, maxStepAttempts: 3 }
escalation: { operatorPort: 4700, interventionTtlMinutes: 30 }
redaction:
  maskReplacement: "****"
  extraSecretEnvPrefixes: ["SCRIBE_SECRET_"]
```

9. Evidence and observability

Serves evaluation criteria: Evidence requirement 3.5; also what makes every other claim verifiable.

Every run - discovery, replay, or escalated - writes one self-contained directory: a structured JSONL event log (what happened and, during discovery, why the model chose it), screenshots at each step, checkpoint, and failure, machine-readable results, and any intervention records. The log is append-only and every entry is attributed to an actor, so a run that crossed a human handoff reads as one continuous, auditable story. All writes pass through the redacting logger (Section 8.4).

```
evidence/                                     # 18 committed run directories
disc_20260912173144/                         # real discovery recording (OpenAI gpt-4o)
  run.jsonl                                  # every event; record shape below
  screenshots/001_... .png                  # per turn + outcome + stuck captures
  artifact.json                             # what the Recorder emitted
  discovery-summary.json
disc_20260912173538/                         # outcome probe: declare_outcome run
  outcome.json                             # merged into the artifact by the CLI
... 5 more disc_* dirs                     # probes + recordings for the other
                                          # two capabilities
replay_20260912173725/                      # happy path (success)
  run.jsonl / result.json / artifact.json / screenshots/
replay_20260912173732/                      # business outcome: MEMBER_NOT_FOUND
replay_20260912173739/                      # injected interstitial -> recovery -> success
replay_20260913015019...45/                 # standing check: Active, Dormant,
                                          # ACCESS_DENIED, MEMBER_NOT_FOUND
replay_20260913015057/                      # risky submit -> approve_once -> escalated,
                                          # finalStatus success (confirmation no.)
replay_20260913015112/                      # validation outcome carried through
                                          # escalation: DEPOSIT_BELOW_MINIMUM
replay_20260913151918/                      # cross-tenant replay on CU North (:4650)
replay_escalation_20260912180848/            # session expiry -> human -> resumed
  interventions.json                       # request + claim + disposition
  run.jsonl                               # actor:"human" entries + control chain

run.jsonl record (as built):
{ seq, ts, actor: "agent"|"replay"|"human"|"operator"|"system",
  type, ...event-specific fields }
// event types include: run_start, act_click/act_fill/act_navigate (with
// intent + target), step_start/step_done, checkpoint evaluations,
// recovery_applied, outcome_declared, net_blocked, control_transition
// (with chainHash), human_action, intervention_resolved, run_finished
```

Eighteen run directories are committed: seven discovery runs (the real recordings plus the outcome probes that declared MEMBER_NOT_FOUND, ACCESS_DENIED, and DEPOSIT_BELOW_MINIMUM), ten replays exercising every arm of the taxonomy - happy paths, both terminal business outcomes, an interstitial recovery, a risky approve_once run, a validation outcome carried through escalation, and a cross-tenant replay on the second tenant app - and one escalated run with a live human hand-off. Beyond the raw record, the telemetry is consumed: npm run health aggregates strategy ranks, recoveries, and durations across evidence/ into a per-capability locator-health drift report (Section 10.3).

10. Heterogeneity and multi-tenant reuse

Serves evaluation criteria: Generalization to the real environment (criterion 5). The multi-tenant overlay is built and demonstrated - a cross-tenant replay is committed evidence; the extra surface drivers remain design, per the brief's instruction.

10.1 The surface seam

One sentence carries the whole abstraction: **the artifact records what to find and verify (semantic descriptors and declarative conditions); the driver owns how to perceive and act.** Nothing in the artifact names a DOM, a selector engine, or a browser - so extending to a new surface means writing a driver, not re-recording capabilities.

```
// src/surface/types.ts (condensed) - the seam, exactly as built
export interface SurfaceDriver {
  launch(): Promise<void>; // fresh session; entry navigation goes through act()
  close(): Promise<void>;

  observe(): Promise<Observation>; // { url, title, nodes: UiNode[], pageText }

  // Replay path: resolve a recorded target through its ranked strategies.
  // Exactly one element must match; ambiguity fails closed.
  resolve(target: StepTarget, opts?): Promise<ResolveOutcome>;

  // The single gate through which every state-changing interaction passes.
  // The PolicyEngine is consulted INSIDE this method (the chokepoint).
  act(req: ActionRequest & { ref?: string }): Promise<ActionResult>;

  synthesizeTarget(ref: string): Promise<TargetSynthesis | null>; // discovery -> artifact

  evalCondition(cond: Condition): Promise<boolean>;
  waitForCondition(cond: Condition, timeoutMs: number): Promise<boolean>;

  // Human-control window: operator clicks/inputs in the live session are
  // reported (password values masked at the source) for the audit trail.
  startHumanCapture(onEvent: (e: HumanActionEvent) => void): Promise<void>;
  stopHumanCapture(): Promise<void>;

  url(): string; title(): Promise<string>; screenshot(absPath: string): Promise<void>;
}

export interface UiNode { // the surface-agnostic observation unit
  ref: string; // ephemeral, valid for this observation only
  role: string; name: string; value?: string;
  disabled?: boolean; checked?: boolean;
  bbox: [number, number, number, number];
  framePath: string[]; // window/frame/pane addressing
}
```

Surface	How the same artifact executes
Modern web (built)	PlaywrightDriver: a11y snapshot for observe; role/label/text engines for resolve; DOM events for act.
Legacy web (built - the mock app is deliberately this)	Same driver - resolution simply lands lower in the strategy ranking (nearText, css, bbox) because roles/labels are sparse. framePath addresses framesets. No schema change.
Desktop (designed)	UIA (Windows) / AXUIElement (macOS) driver: the OS accessibility tree maps onto the same UiNode shape - the glossary itself notes a11y trees exist on desktop. role+name strategies carry over; css ranks are simply skipped.
Screenshot-only / hostile (designed)	Vision driver: observe via OCR + detection into UiNode[]; resolve uses elementDescription plus the recorded bbox rank (captured on every targeted step for exactly this); act by coordinates. A model assists perception only - decisions stay deterministic, and the policy chokepoint is unchanged.

10.2 Multi-tenant reuse: base artifact + tenant binding overlay (built)

Hundreds of institutions run the same vendor product, differently branded and versioned. Re-recording per tenant is the failure mode to avoid, so the unit of recording is the **vendor product** (base artifact), and each tenant contributes only a thin, reviewable **binding**. This is no longer just a schema - below is the committed tenants/cu-north.json that the cross-tenant demo runs through (description re-wrapped as a comment):

```

{
  "schemaVersion": "1.0",
  "tenantId": "cu-north",
  "appId": "cu-backoffice", // must match the base artifact, or the merge refuses
  // description (re-wrapped): CU North runs the same teller product re-branded
  // as "CU North TellerWorks" - green skin, tw_* auto-generated ids, and
  // member->customer vocabulary. Routes, forms, and business behavior are
  // identical, so the recorded artifacts replay here through this overlay
  // alone - no re-recording.
  "requiredOrigins": ["http://localhost:4650"],
  "appFingerprint": {
    "titlePattern": "CU North TellerWorks",
    "markers": ["Operator Log On"]
  },
  "vocabulary": {
    "Teller Sign-In": "Operator Log On",
    "Teller ID": "Operator ID",
    "Passcode": "PIN",
    "Sign In": "Log On",
    "Teller Desk": "Operator Desk",
    "Member Search": "Customer Search",
    "Member Number": "Customer Number",
    "Search": "Find",
    "Member Results": "Customer Results",
    "Member Profile": "Customer Profile",
    "Member No": "Customer No",
    "Member Since": "Customer Since",
    "No records found": "No matching customers on file",
    "No records found.": "No matching customers on file."
  }
}

```

- **Merge at load, never mutate the base:** replay --tenant cu-north resolves base + binding into an effective artifact in memory, and the merged result is re-validated through the same strict artifact schema - a bad overlay cannot smuggle in an invalid capability. One upgrade path, many tenants.
- **Vocabulary is exact-match on whole strings,** applied to role names, labelText/nearText values, textPresent detectors, and element descriptions - never to css, bbox, urlMatches, step values, or extract patterns. "Member Search" -> "Customer Search" cannot mangle "Member Number", and mechanics stay untouched: only identification vocabulary shifts.
- **stepOverrides replace a step's strategy list wholesale** - no partial patching to reason about - and vocabulary deliberately skips overridden steps: an override is already tenant-final.
- **Bindings carry no flow logic:** a tenant cannot fork behavior, only identification and configuration. A tenant needing a different flow is a new capability version, reviewed like any other.
- **Fails closed:** an appld mismatch refuses the merge (never across products); unknown step ids are errors; and without --tenant the artifact's requiredOrigins still pin tenant A, so tenant B's origin is denied at the chokepoint. test/integration/tenant.test.ts proves both directions.

The committed cross-tenant run (evidence/replay_20260913151918) replays the artifact recorded on tenant A against CU North through this overlay alone - no re-recording, no LLM. Its telemetry shows every step resolving at rank 0 (role/labellText/nearText): tenant A's ctI00_* css ranks never had to match the tw_* markup. That is the semantic-first locator thesis (Section 5) doing exactly the work it was designed for.

10.3 Detecting and managing drift

- **Telemetry as sensor (built):** every replay records, per step, which strategy rank matched (0 = primary), the strategy kind, attempts, recoveries applied, and duration - committed in every result.json. npm run health aggregates the evidence corpus into a per-capability locator-health report; --ci exits non-zero when a capability is drifting or broken. The committed standing-check runs already show a real drift signature: replayed against a different member than it was recorded on, step s8 falls from the recorded-name rank to the parameterized nearText rank - visible in telemetry long before anything breaks.

- **Fail fast on version drift (built):** the appFingerprint preflight stops the run before any action when the app is not what was recorded; the tenant overlay replaces the fingerprint wholesale for exactly this reason.
- **Repair loop (design):** degraded capabilities get a canary re-validation; a bounded, policy-checked LLM pass may propose a locator patch as a diff for human review (the brief's assisted-fallback stretch) - the model proposes, a person approves, replay stays deterministic.

11. The mock target application ("CU BackOffice")

Serves evaluation criteria: Enables the robustness, safety, and escalation demonstrations; stands in for the real environment per brief Section 4.

A small Express + EJS server-rendered app with deliberately hostile, legacy-style markup: nested table layouts, no test IDs, generated-looking IDs (#ctl00_MbrSrch_txtNum), font-tag styling, sparse label associations, and inline styles. Synthetic data only, held in memory - a successful sub-account open really mutates it (appends the account, increments the confirmation counter); restarting resets the seed. It runs offline; reviewers never need external services (an LLM key is needed only for discovery). The same server also ships the tenant-B skin: npm run target:north serves "CU North TellerWorks" on port 4650 with green branding, tw_* ids, and customer vocabulary (Section 10.2).

11.1 Flows

- **Login** - teller credentials (synthetic), session cookie; expiring the session mid-flow is an injectable fault.
- **Member search** - by member number; not-found is a natural state ("No records found.").
- **Member detail** - name, standing, member-since, and balances inside nested tables; member 66666 is teller-restricted and renders an access-denied screen - a natural state, no injection needed.
- **Open sub-account** - multi-field form; deposits under \$5.00 are rejected with a validation message (the DEPOSIT_BELOW_MINIMUM outcome); the mutating confirm issues a confirmation number - the risky-action demo.

11.2 Fault-injection matrix (armed one-shot via GET /__faults?arm=...)

The replay CLI's --inject flag arms exactly one fault before the run; each fault clears itself the moment it fires, so a recovery's retry sees the healthy app again - which is what makes bounded recovery meaningful rather than a tight loop against a permanently broken page.

Injection	App behavior	Exercises	System response (as demonstrated)
(no injection) unknown member id	Empty result grid + "No records found."	Business outcome	MEMBER_NOT_FOUND returned as a value
(no injection) deposit under \$5.00	Form re-renders with a validation message	Business outcome	DEPOSIT_BELOW_MINIMUM, terminal - carried inside the escalated wrapper when the run also had an approval
(no injection) restricted member 66666	Access-denied screen	Business outcome	ACCESS_DENIED, terminal
interstitial	Maintenance modal covers the page	Recoverable	Declared dismiss recovery clicks OK; step retried; recovery_applied logged
slow	"Loading, please wait" placeholder page	Recoverable	Declared waitRetry recovery: 2500 ms backoff, max 3 attempts, inside step budgets
session-expiry	Session cookie invalidated on the next request	Recoverable -> escalate	A declared runSteps re-login can absorb it; the committed lookup leaves it undeclared, so the checkpoint fails -> intervention -> human re-authenticates in the live browser -> completed_step -> resume
error500	Server error page (one-shot)	Hard failure	Evidence bundle captured; structured hard_failure returned

Three capabilities are recorded against the app and committed: **member-savings-lookup** (read-only; the real-LLM discovery run), **member-standing-check** (two typed outputs; one artifact yields three terminal shapes - success,

ACCESS_DENIED, MEMBER_NOT_FOUND), and **subaccount-open** (riskLevel mutating: every replay pauses at the confirm submit for a signed approval, and a too-small deposit surfaces DEPOSIT_BELOW_MINIMUM through the same escalated wrapper). The tenant-B skin is not a stretch note: it is built, served by npm run target:north, and exercised by the committed cross-tenant replay of Section 10.2.

12. Testing strategy - where it counts

Serves evaluation criteria: Code quality (criterion 7): "tested where it counts", not blanket coverage.

Layer	What is tested and why it is the right thing
Unit: artifact schema + conditions (core.test.ts)	Zod round-trips, rejection of malformed artifacts, {{...}} template rendering, condition-AST evaluation - the contract everything else trusts.
Unit: policy + redaction (policy.test.ts)	Allow/deny matrix over origins, action kinds, risky gates and approval states; the redactor: registered secrets and their URL-encoded variants never survive into log output - the safety claims, verified.
Unit: deviation classifier (classifier.test.ts)	Precedence (outcome beats recovery beats failure), bounded retries, fail-closed on unknown states - the taxonomy the rubric weighs.
Unit: recorder (recorder.test.ts)	Literal values -> {{inputs.x}} parameterization, secret canonicalization, artifact distillation from the recorded action log.
Unit: escalation + signing (escalation.test.ts, signing.test.ts)	Intervention lifecycle (pending -> claimed -> resolved; TTL expiry only fires while still pending), disposition rules, HMAC payload binding, and verifyResolutionForRecord tamper checks.
Unit: tenant merge (tenant.test.ts)	Exact-match vocabulary, mechanics never rewritten, wholesale step overrides, applId-mismatch refusal, re-validation of the merged artifact.
Unit: locator health (locator-health.test.ts)	Drift aggregation over telemetry: rank histograms and the drifting/broken classification behind npm run health --ci.
Integration: 6 suites (vitest + Playwright against live app instances)	replay.test.ts covers the Section 11.2 taxonomy rows end-to-end; driver.test.ts covers ranked resolution, ambiguity fail-closed, and the act() chokepoint; escalation.test.ts walks pause -> claim -> signed disposition -> resume with a scripted operator; discovery.test.ts drives the real loop with a scripted provider double; backstop.test.ts proves the network layer kills exfiltration and redirect chains in-flight; tenant.test.ts proves the overlay and the fail-closed default.

93 tests across these 14 files, all green. npm test needs no API key and no pre-started services: integration suites boot their own target-app instances. Three gates run in order: npm run typecheck (strict TS), npm run lint (ESLint plus scripts/depcheck.mjs, which fails the build if replay/ ever imports llm/ - Section 6's determinism enforced structurally), npm test.

Deliberately untested: EJS templates, CLI argument parsing, console styling - low-risk glue. The live-model path is the one real discovery run committed as evidence (OpenAI gpt-4o, 9 turns, evidence/disc_20260912173144).

13. Repository layout and deliverables mapping

Serves evaluation criteria: Communication (criterion 8) and the brief's exact deliverable paths.

```

/                                     # public GitHub repo root
README.md                           # setup, keys, demo path (exact commands)
REPORT.md                           # the 7 required headings
policy.yaml                         # global allowlist and budgets (Appendix C)
.env.example                        # LLM key (discovery only) + demo secrets
capabilities/                       # the three committed artifacts (reviewable JSON)
tenants/cu-north.json              # tenant-B binding overlay (Section 10.2)
evidence/                          # 18 committed runs: disc_*, replay_* (Section 9)
docs/                              # this document (and its generator)
scripts/                           # auto-approve.ts, capture-escalation.ts,
                                   #   depcheck.mjs (the import-boundary gate)

src/
  core/ surface/ llm/ agent/ replay/
  policy/ escalation/ evidence/ cli/
  target-app/                       # mock CU back-office, both tenant skins
test/
  fixtures/ unit/ integration/

```

REPORT.md uses the seven mandated headings verbatim - Architecture, Artifact schema, Determinism and error handling, Heterogeneity and multi-tenant, Escalation and handoff, Safety, Cuts - each a distilled page-share of this document's Sections 3, 5, 6, 10, 7, 8, and 14 respectively. Replay runs with no API key; only discovery needs one, and README says so up front.

13.1 Demo path (condensed from README)

```

npm install && npx playwright install chromium
npm run target                # mock app on http://localhost:4600

# 1. DISCOVERY (uses an LLM key once): goal in, capability artifact out
npm run discover -- \
  --goal "Look up the member by member number and read their savings account balance" \
  --entry http://localhost:4600/login \
  --id member-savings-lookup --name "Member savings balance lookup" \
  --app cu-backoffice --input memberId=12345 --provider openai
# -> capabilities/member-savings-lookup.json (reviewStatus: draft)

# 2. Review + approve (gates unattended replay of risky capabilities)
npm run approve -- --capability member-savings-lookup --reviewer you

# 3. DETERMINISTIC REPLAY (no LLM key): typed params in, typed outputs out
npm run replay -- --capability member-savings-lookup --param memberId=12345
# -> success { savingsBalance: "$1,204.55" } + locator-rank telemetry
npm run replay -- --capability member-savings-lookup --param memberId=99999
# -> business_outcome MEMBER_NOT_FOUND (a declared answer, not an error)
npm run replay -- --capability member-savings-lookup --param memberId=12345 \
  --inject interstitial          # -> declared recovery dismisses the dialog

# 4. ESCALATION: one-shot session expiry -> checkpoint fails -> intervention
npm run replay -- --capability member-savings-lookup --param memberId=12345 \
  --inject session-expiry --headed
# open the tokened console URL the CLI prints: claim -> re-authenticate in
# the headed browser (same live session) -> resolve "completed_step"
# -> engine re-verifies the checkpoint and resumes -> escalated / success

# 5. Multi-tenant: the same artifact on re-skinned tenant B (Section 10.2)
npm run target:north           # tenant B on http://localhost:4650
npm run replay -- --capability member-savings-lookup --tenant cu-north \
  --param memberId=12345       # -> success, every step at rank 0

npm run catalog                # capabilities as an agent-facing summary
npm run health                 # locator-health drift report (--ci gate)

```

The README additionally walks the mutating subaccount-open demo (pause at the risky submit -> approve_once -> confirmation number; a \$2.00 deposit surfaces DEPOSIT_BELOW_MINIMUM instead), the member-standing-check drift demo, and two scripted operators (scripts/capture-escalation.ts, scripts/auto-approve.ts) that drive the same console HTTP API a human would - the identical control-transfer path, no human required.

14. Implementation roadmap as executed, cut lines, and stretch goals

Serves evaluation criteria: Everything - this is the order in which the criteria were built and verified.

14.1 Phases (as planned - and as executed; est. 28-36 focused hours)

P h	Build	Exit criterion (definition of done)	Hrs
0	Scaffold: strict TS, ESLint + dependency-boundary rule, vitest, prettier	CI-green empty skeleton; replay/ -> llm/ import fails lint	1-2
1	core/: artifact schema, condition AST, result contract, policy config (Zod)	Appendix A artifact parses; malformed fixtures rejected; types inferred	3-4
2	target-app/: flows, legacy markup, fault injection, seed data	All Section 11.2 states reachable by hand in a browser	3-4
3	surface/: PlaywrightDriver - a11y observe, ranked resolve, act with policy chokepoint, screenshots	Driver test suite green incl. frames and ambiguity fail-closed	4-5
4	policy/ + evidence/: engine, redacting JSONL logger, run dirs	Policy matrix tests green; secret-scan test proves no leaks	2-3
5	llm/ + agent/: both providers, discovery loop, stop conditions, Recorder with parameterization + probe runs	Real discovery completes the lookup goal; artifact emitted and valid	5-6
6	replay/: executor, waits, checkpoints, classifier, recoveries, result contract	All taxonomy integration tests green; no-API-key replay verified	5-6
7	escalation/: RunController, intervention store, operator console, human-action recorder, resume	session-expiry scenario: pause -> human login in same browser -> resume -> success, fully evidenced	4-5
8	cli/ + evidence capture: the committed evidence set (18 runs); README demo path verified from a clean clone	A reviewer can reproduce replay demos with three commands	2-3
9	REPORT.md (7 headings), README polish, final read-through	Docs match behavior; cut lines explicit	2-3

All ten exit criteria hold in the committed repo (93/93 tests, the evidence set, no-key replay). The one piece of work the plan did not anticipate: the **network-layer backstop** (Section 8.5), added when implementation demonstrated that the act() chokepoint alone cannot govern traffic the page initiates by itself - the committed redirect-chain tests are the record of that reversal.

14.2 Cut lines - deliberate, with the seam designed

- **Operator console is minimal by intent** (list, claim, resolve - token-authenticated, dispositions HMAC-signed): the brief scopes out a real co-browsing console; the control-transfer model is the graded part and it is fully real.
- **Remote session streaming (CDP screencast / noVNC)**: designed in Section 7.4, not built - the local headed browser makes the handoff real without transport work.
- **Desktop and vision drivers**: interface + design only (Section 10.1); the brief says design, not build.
- **Assisted LLM recovery on replay failure**: designed as a propose-a-patch flow (Section 10.3), not built - keeps the replay path pure.
- **Shared-secret HMAC and a single operator identity** instead of per-operator OIDC + KMS-held asymmetric keys: scope cut documented in Section 7.6 with the production path named.

14.3 Stretch goals - both attempted, both landed

- **1. Agent-facing capability catalog (built):** npm run catalog lists the committed artifacts as agent-facing contract summaries - the contract band of Section 5 (typed inputs/outputs, outcomes, risk and review state) without the mechanics - closing the brief's through-line: the artifact as an agent-invocable capability.
- **2. Tenant-B variant (built):** the CU North TellerWorks skin plus tenants/cu-north.json replay the tenant-A-recorded artifact unchanged on tenant B (committed run replay_20260913151918) - Section 10 is a demonstration, not a promise.

14.4 What gets built next with more time

- Prevention-grade redirect interception via CDP Fetch.requestPaused, upgrading the Section 8.5 watchdog's detect-and-kill residual to true prevention.
- Operator identity and keys: per-operator OIDC on the console; asymmetric, KMS-held signing keys replacing the shared HMAC secret (Section 7.6).
- A stability harness (on the order of 50 replays per capability per app version) plus canary re-validation feeding the locator-health report, and confidence scoring gating unattended runs.
- The assisted-fallback repair loop: bounded LLM-proposed locator patches, shipped as reviewed diffs (Section 10.3).
- A signed capability registry: approval workflow, distribution, revocation.
- DLP-style scanning of extracted outputs, beyond declared-sensitive masking.
- Desktop (UIA/AX) and vision drivers behind the same SurfaceDriver seam; remote operator transport (CDP screencast / noVNC).
- OpenTelemetry export of the run/step telemetry that today lands in result.json.

15. Risk register

Risk	L	I	Mitigation / outcome
Discovery flakiness (model meanders, mislabels targets)	M	M	Closed four-tool action set, compact numbered snapshots, loop detector, declare_outcome grounded against on-screen text; only a verified successful run is distilled, and retry runs are cheap. The committed recording took one 9-turn run.
Hostile markup yields poor accessible names	M	M	nearText/bbox ranks exist for exactly this; the mock app keeps visible text anchors (as real back-office apps do). Observed in practice: the standing-check drift lands on the parameterized nearText rank - degraded, not broken.
Page-initiated traffic bypasses the act() chokepoint (exfil subresources, redirect chains)	H	H	Materialized during implementation and addressed: the network backstop (Section 8.5) aborts off-allowlist requests, vets first-hop redirects, and a watchdog kills the context on later hops. Residual: a later hop's egress races the kill - detection is guaranteed; prevention is the CDP upgrade (14.4).
Human-action recording rabbit hole	M	L	Scope pinned to clicks/inputs/submits/navigations via one injected binding (startHumanCapture), password values masked at the source; anything deeper is out of scope and documented.
Handoff race conditions (agent acts while human drives)	L	H	Control ownership is a single in-process ledger with legal-transition checks and a hash-chained audit trail; the executor blocks while HUMAN holds control, and every transition is a logged control_transition event carrying the chain hash.
Sensitive data slips into evidence	L	H	Redaction at the logger boundary (single write path, URL-encoded variants included), secret-scan tests, parameterized artifacts; the committed result.json files show the masked savingsBalance.
Scope creep vs the time box	M	M	Phase exit criteria; cut lines pre-declared; stretch goals gated on a finished core - and landed only after the suite was green.

L = likelihood, I = impact (H/M/L).

Appendix A - The committed artifact (capabilities/member-savings-lookup.json)

Recorded by the real gpt-4o discovery run, approved via npm run approve, and executed by the committed replay evidence - content verbatim, JSON whitespace compacted to fit the page (the file pretty-prints each strategy across multiple lines). Internally consistent with Sections 5, 6, and 11 because it is not an example: it is the artifact.

```
{
  "schemaVersion": "1.0",
  "capability": {
    "id": "member-savings-lookup",
    "version": "1.0.0",
    "name": "Member savings balance lookup",
    "description": "Look up the member by member number and read their savings account balance"
  },
  "target": {
    "appId": "cu-backoffice",
    "surface": "web",
    "entrypoint": "{{env.APP_BASE_URL}}/login",
    "appFingerprint": { "titlePattern": "CU BackOffice -", "markers": [] }
  },
  "inputs": [
    { "name": "memberId", "type": "string", "required": true, "pattern": "^\\d+$",
      "sensitive": false, "example": "12345" }
  ],
  "outputs": [
    { "name": "savingsBalance", "type": "money", "sensitive": true, "sourceStep": "s9" }
  ],
  "steps": [
    { "id": "s1", "intent": "Open the application entrypoint", "action": "navigate",
      "url": "{{env.APP_BASE_URL}}/login", "sensitive": false,
      "checkpoint": { "urlMatches": "/login" }, "risk": "safe" },
    { "id": "s2", "intent": "Enter teller username", "action": "fill",
      "target": {
        "strategies": [
          { "kind": "role", "role": "textbox", "name": "Teller ID" },
          { "kind": "labelText", "value": "Teller ID" },
          { "kind": "nearText", "value": "Teller ID" },
          { "kind": "css", "value": "#ctl00_LoginCtl_txtUser" },
          { "kind": "bbox", "value": [603, 135, 139, 21] }
        ],
        "elementDescription": "textbox \"Teller ID\"", "framePath": []
      },
      "value": "{{secrets.tellerUsername}}", "sensitive": true, "risk": "safe" },
    { "id": "s3", "intent": "Enter teller password", "action": "fill",
      "target": {
        "strategies": [
          { "kind": "role", "role": "textbox", "name": "Passcode" },
          { "kind": "labelText", "value": "Passcode" },
          { "kind": "nearText", "value": "Passcode" },
          { "kind": "css", "value": "#ctl00_LoginCtl_txtPass" },
          { "kind": "bbox", "value": [603, 168, 139, 21] }
        ],
        "elementDescription": "textbox \"Passcode\"", "framePath": []
      },
      "value": "{{secrets.tellerPassword}}", "sensitive": true, "risk": "safe" },
    { "id": "s4", "intent": "Sign in to teller system", "action": "click",
      "target": {
        "strategies": [
```

```

    { "kind": "role", "role": "button", "name": "Sign In" },
    { "kind": "css", "value": "#ctl00_LoginCtl_btnGo" },
    { "kind": "bbox", "value": [603, 201, 58, 21] }
  ],
  "elementDescription": "button \"Sign In\"", "framePath": []
},
"sensitive": false, "checkpoint": { "urlMatches": "/desk" }, "risk": "safe" },
{ "id": "s5", "intent": "Navigate to member search", "action": "click",
  "target": {
    "strategies": [
      { "kind": "role", "role": "link", "name": "Member Search" },
      { "kind": "css", "value": "#ctl00_Nav_lnkSearch" },
      { "kind": "bbox", "value": [84, 34, 89, 16] }
    ]
  },
  "elementDescription": "link \"Member Search\"", "framePath": []
},
"sensitive": false, "checkpoint": { "urlMatches": "/members" }, "risk": "safe" },
{ "id": "s6", "intent": "Input member number for search", "action": "fill",
  "target": {
    "strategies": [
      { "kind": "nearText", "value": "Member Number" },
      { "kind": "css", "value": "#ctl00_MbrSrch_txtNum" },
      { "kind": "bbox", "value": [128, 106, 97, 21] }
    ]
  },
  "elementDescription": "textbox near \"Member Number\"", "framePath": []
},
"value": "{{inputs.memberId}}", "sensitive": false, "risk": "safe" },
{ "id": "s7", "intent": "Perform member search", "action": "click",
  "target": {
    "strategies": [
      { "kind": "role", "role": "button", "name": "Search" },
      { "kind": "nearText", "value": "Member Number" },
      { "kind": "css", "value": "#ctl00_MbrSrch_btnFind" },
      { "kind": "bbox", "value": [237, 106, 58, 21] }
    ]
  },
  "elementDescription": "button \"Search\"", "framePath": []
},
"sensitive": false, "risk": "safe" },
{ "id": "s8", "intent": "Access member details for Alvarez, Maria", "action": "click",
  "target": {
    "strategies": [
      { "kind": "role", "role": "link", "name": "Alvarez, Maria" },
      { "kind": "nearText", "value": "{{inputs.memberId}}" },
      { "kind": "css", "value": "#ctl00_MbrGrd_r0_c1 > font > a" },
      { "kind": "bbox", "value": [274, 201, 80, 16] }
    ]
  },
  "elementDescription": "link \"Alvarez, Maria\"", "framePath": []
},
"sensitive": false,
"checkpoint": { "urlMatches": "/members/{{inputs.memberId}}" }, "risk": "safe" },
{ "id": "s9", "intent": "Record the savings account balance of the member",
  "action": "extract",

```

```

    "target": {
      "strategies": [
        { "kind": "nearText", "value": "Savings" },
        { "kind": "css", "value": "#ctl00_AcctGrd_r0_c2 > font" },
        { "kind": "bbox", "value": [640, 302, 57, 16] }
      ],
      "elementDescription": "font near \"Savings\"", "framePath": []
    },
    "sensitive": false, "outputName": "savingsBalance",
    "extractPattern": "\\$[\\d,]+\\.\\d{2}", "risk": "safe" }
  ],
  "outcomes": [
    { "code": "MEMBER_NOT_FOUND", "terminal": true,
      "description": "The member with ID 99999 could not be found in the system.",
      "detector": { "textPresent": "No records found." } }
  ],
  "recoveries": [
    { "id": "maintenance-interstitial", "appliesTo": "global",
      "detector": { "elementVisible": { "kind": "role", "role": "dialog", "name": "System Notice" } },
      "action": { "kind": "dismiss", "target": {
        "strategies": [
          { "kind": "role", "role": "button", "name": "OK" },
          { "kind": "css", "value": "#ctl00_Notice_btnOk" }
        ],
        "elementDescription": "OK button on the maintenance notice dialog", "framePath": []
      } },
      "maxAttempts": 2 },
    { "id": "transient-slow-load", "appliesTo": "global",
      "detector": { "textPresent": "Loading, please wait" },
      "action": { "kind": "waitRetry", "backoffMs": 2500 }, "maxAttempts": 3 }
  ],
  "successCheckpoint": { "urlMatches": "/members/{{inputs.memberId}}" },
  "policy": {
    "requiredOrigins": ["http://localhost:4600"],
    "allowedActionKinds": ["navigate", "fill", "click", "extract"],
    "riskLevel": "readonly",
    "unattendedReplay": true
  },
  "provenance": {
    "provider": "openai",
    "model": "gpt-4o",
    "runId": "disc_20260912173144",
    "recordedAt": "2026-09-12T17:31:59.793Z",
    "reviewStatus": "approved",
    "reviewedBy": "operator-jsmith",
    "reviewedAt": "2026-09-12T17:36:57.490Z"
  }
}

```

Appendix B - Result contract examples (committed runs)

Real committed result.json files, trimmed where marked. The success run shows savingsBalance as "****" because the output is declared sensitive and redaction happens before anything reaches disk; the live terminal prints the real value.

```
// evidence/replay_20260912173725/result.json - SUCCESS (telemetry trimmed)
{
  "runId": "replay_20260912173725",
  "mode": "replay",
  "capabilityId": "member-savings-lookup",
  "capabilityVersion": "1.0.0",
  "params": { "memberId": "12345" },
  "startedAt": "2026-09-12T17:37:25.176Z",
  "finishedAt": "2026-09-12T17:37:26.601Z",
  "result": {
    "status": "success",
    "outputs": { "savingsBalance": "****" } // sensitive -> masked before disk;
  }, // the live terminal prints the value
  "telemetry": [
    { "stepId": "s1", "strategyRank": null, "attempts": 1,
      "recoveriesApplied": [], "durationMs": 95 }, // navigate: untargeted
    { "stepId": "s2", "strategyRank": 0, "strategyKind": "role", "attempts": 1,
      "recoveriesApplied": [], "durationMs": 89 }
    // ... s3-s9 identical shape; every step resolved at rank 0 (role/nearText)
  ],
  "evidenceDir": "evidence/replay_20260912173725"
}

// evidence/replay_20260912173732/result.json - BUSINESS OUTCOME (result block;
// its telemetry ends at s7, where the detector legitimately stopped the run)
"result": {
  "status": "business_outcome",
  "code": "MEMBER_NOT_FOUND",
  "description": "The member with ID 99999 could not be found in the system.",
  "extracted": {}
}

// evidence/replay_20260913015057/result.json - ESCALATED risky run (excerpt)
"params": { "memberId": "12345", "accountType": "Checking",
  "nickname": "Rainy Day", "depositAmount": "40.00" },
"result": {
  "status": "escalated",
  "interventions": [
    { "id": "iv_replay_20260913015057_1",
      "reason": "capability policy forbids unattended risky actions",
      "type": "approval",
      "disposition": "approve_once",
      "operator": "operator-jsmith" }
  ],

```

```
  "finalStatus": "success", // the wrapped underlying result
  "outputs": { "confirmationNumber": "CU-2026-4182" }
}

// HARD FAILURE - the remaining arm of the union. No committed demo run ends
// this way (recoveries or escalation absorb the injected faults); the shape
// from src/core/result.ts, illustrative values:
{
  "status": "hard_failure",
  "error": {
    "stepId": "s7",
    "intent": "Perform member search",
    "expected": "urlMatches(/members)",
    "observed": "checkpoint not satisfied after 3 attempts",
    "evidence": ["evidence/replay_x/screenshots/007_failure.png"]
  }
}
}
```

Appendix C - policy.yaml

```
# policy.yaml as committed (comments re-wrapped to fit the page).
# Global allowlist and budgets. Deny by default: anything not listed here is
# refused at the SurfaceDriver chokepoint, regardless of what the model asks.
allowlist:
  origins:
    - http://localhost:4600 # mock CU back-office - tenant A (Community One)
    - http://localhost:4650 # same product re-skinned - tenant B
                          #   (CU North, via tenants/cu-north.json)

actionKinds: [navigate, click, fill, select, press, extract]

risky:
  discoveryRequiresConfirmation: true # a human approves every mutating
                                     #   action while recording
  unattendedRequiresApprovedArtifact: true # replay may auto-run risky steps
                                           #   only for reviewed artifacts

budgets:
  discoveryMaxTurns: 40
  stepTimeoutMs: 15000
  runTimeoutMs: 300000
  maxStepAttempts: 3

escalation:
  operatorPort: 4700
  interventionTtlMinutes: 30

redaction:
  maskReplacement: "****"
  extraSecretEnvPrefixes: ["SCRIBE_SECRET_"]
```